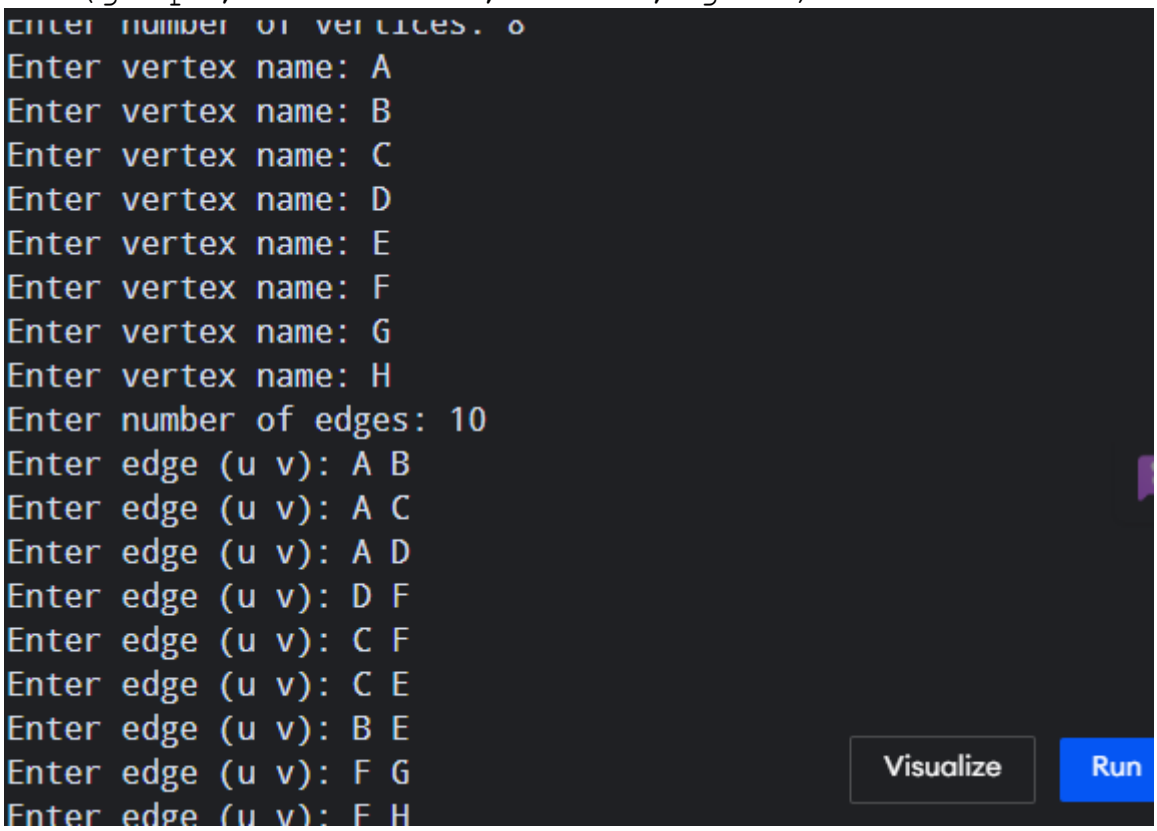
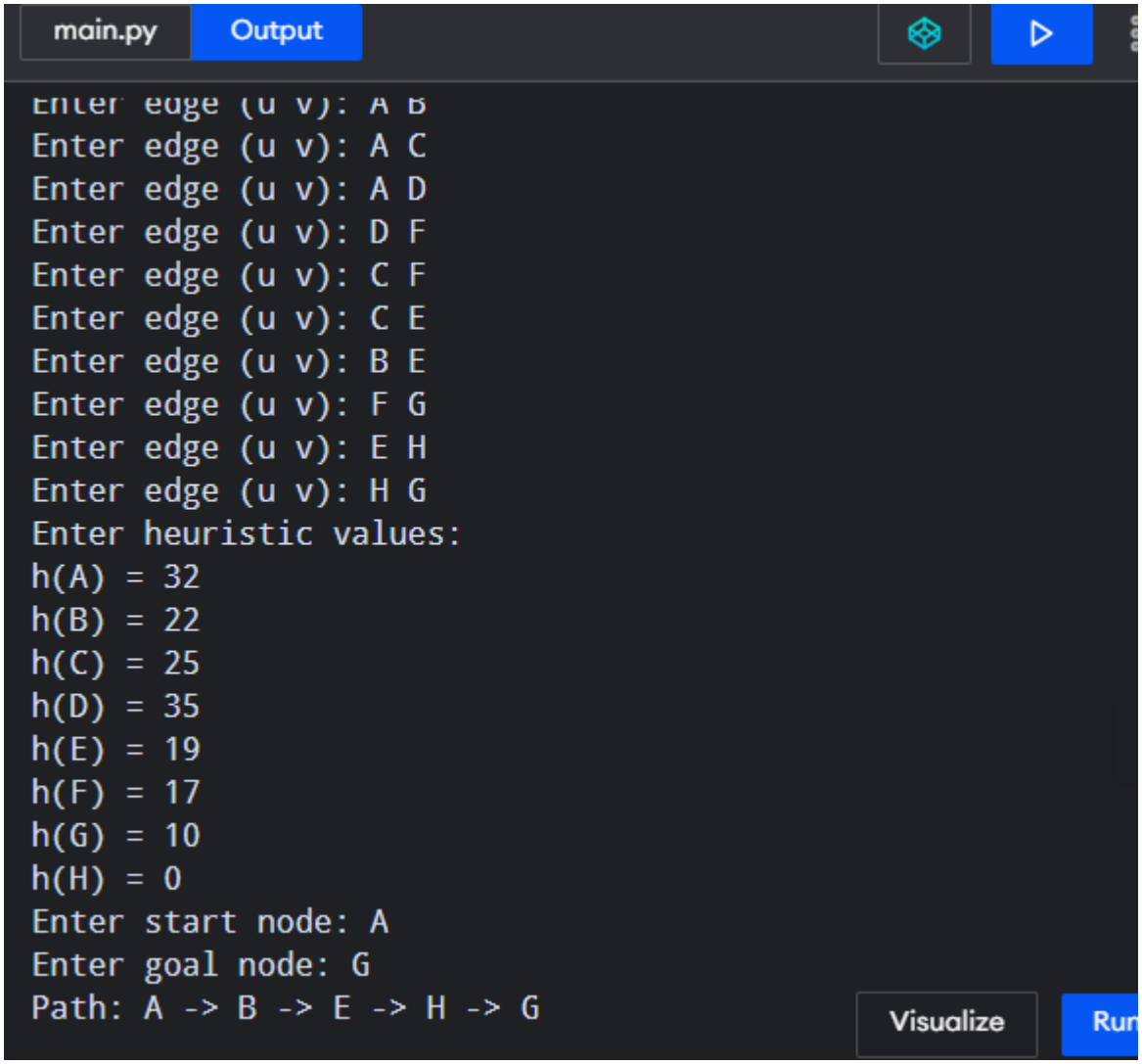
G-BFS ALGORITHM

```
import heapq
def gbfs(graph, heuristic, start, goal):
    queue = [(heuristic[start], start)]
    visited = set()
    parent = {start: None}
    while queue:
        h, node = heapq.heappop(queue)
        if node in visited:
            continue
        visited.add(node)
        if node == goal:
            path = []
            while node is not None:
                path.append(node)
                node = parent[node]
            path.reverse()
            print("Path:", " -> ".join(path))
            return
        for neighbor in graph[node]:
            if neighbor not in visited and
neighbor not in parent:
                parent[neighbor] = node
                heapq.heappush(queue,
(heuristic[neighbor], neighbor))
    print("No path found!")
graph = {}
n = int(input("Enter number of vertices: "))
for i in range(n):
    vertex = input("Enter vertex name: ")
    graph[vertex] = []
e = int(input("Enter number of edges: "))
for i in range(e):
    u, v = input("Enter edge (u v): ").split()
    graph[u].append(v)
    graph[v].append(u)
heuristic = {}
```

```
print("Enter heuristic values:")
for vertex in graph:
    heuristic[vertex] = float(
        input("h(" + vertex + ") = ")
    )
start = input("Enter start node: ")
goal = input("Enter goal node: ")
gbfs(graph, heuristic, start, goal)
```




main.py
Output
Enter edge (u v): A B
Enter edge (u v): A C
Enter edge (u v): A D
Enter edge (u v): D F
Enter edge (u v): C F
Enter edge (u v): C E
Enter edge (u v): B E
Enter edge (u v): F G
Enter edge (u v): E H
Enter edge (u v): H G
Enter heuristic values:
h(A) = 32
h(B) = 22
h(C) = 25
h(D) = 35
h(E) = 19
h(F) = 17
h(G) = 10
h(H) = 0
Enter start node: A
Enter goal node: G
Path: A -> B -> E -> H -> G
Visualize
Run